

Type Casting in Java ?

Type casting is nothing but converting one data type into another data type.

There are two types of type casting:

- 1) Implicit OR Widening type casting.
- 2) Explicit OR Narrowing type casting.

Implicit OR Widening type casting.

Java will automatically convert smaller data type to bigger data type. There is no data loss.

Example:

```
byte b = 12;
short s = b;
```

Implicit.java

```
void main()
{
    byte b = 125;
    short s = b; //Implicit OR Widening
    IO.println(s);
}
```

Explicit OR Narrowing type casting.

Converting a bigger data type into a smaller type manually. In this there is a chance of data loss.

Example:

```
short s = 12;
byte b = (byte) s;
```

//Assigning bigger data type value to smaller data type

```
void main()
{
    short s = 0;
    byte b = (byte) s; //Explicit OR Narrowing
    IO.println(b);
}
```

= Assignemt Operator will support WIDENING + NARROWING

method calling will support only WIDENING

How to find out the output in case explicit OR Narrowing casting, if the value is out of range:

byte Case:

Case 1: If the value is within the range -128 to 127 then same output will be generated by the using widening support of java compiler

Case 2: If the value is > 127 then subtract 256 from the given number till the result value is not coming within the range

Case 3: If the value is < -128 then add 256 in the given number till the value is not coming within the range

Explicit.java

```
void main()
{
    byte b = (byte)130;
    IO.println(b); //-126

    byte c = (byte) 190;
    IO.println(c);

    byte d = (byte) 256;
    IO.println(d);

    byte e = (byte) -150;
    IO.println(e);

    byte f = (byte) 590;
    IO.println(f);
}
```

Short type:

Case 1: If the value is within the range -32768 to 32767 then same output will be generated by the using widening support of java compiler

Case 2: If the value is > 32767 then subtract 65536 from the given number till the value is not coming within the range

Case 3: If the value is < -32768 then add 65536 in the given number till the value is not coming within the range

ShortDemo.java

```
void main()
{
    short s = (short) 35000; //65536
    IO.println(s);

    short y = (short)-33000;
    IO.println(y);
}
```

Casting.java

```
void main()
{
    byte x = (byte)18L;
    IO.println("x value = "+x);

    long l = 29L;
    IO.println("l value = "+l);

    int y = (int) 1L;
    IO.println("y value = "+y);
}
```

Is Java pure object oriented language or not ?

No, Java is not a pure object oriented language because it is accepting **primary data type**, Actually any language which accepts primary data type is not a pure object oriented language.

Only Objects are moving in the network but not the primary data type so java has introduced Wrapper class concept to convert the primary data types into corresponding wrapper object.

Primary Data type		Wrapper Object
byte	:	Byte
short	:	Short
int	:	Integer
long	:	Long
float	:	Float
double	:	Double
char	:	Character
boolean	:	Boolean